

Xây dựng và đánh giá cơ chế điều phối Continuous Training có kiểm chứng suy giảm với ngân sách nhân hạn chế trong MLOps

Đề cương khóa luận tốt nghiệp – phạm vi thực hiện 5 tháng

Cập nhật ngày 07/09/2026

Tóm tắt. Đề tài nghiên cứu cơ chế điều phối Continuous Training khi dữ liệu đầu vào thay đổi nhưng chỉ một phần dữ liệu production có nhãn. Evidently, CBPE và ATC là các thành phần giám sát, ước lượng được kế thừa; trọng tâm là chuyển bằng chứng thành quyết định tái huấn luyện có thể thực thi, truy vết và phục hồi khi tác vụ lỗi. Hệ thống kiểm chứng bằng nhãn thật trong ngân sách, chốt snapshot, huấn luyện ứng viên và đánh giá trước khi thay mô hình. Backend production dùng Argo Workflows và Kubeflow Training Operator trên AWS/K3s; Terraform cấp phát hạ tầng, Ansible tự động hóa bootstrap, Prometheus/Grafana giám sát vận hành. Docker chỉ dùng kiểm thử local. Phạm vi gồm hai dataset nhị phân tabular, một model family và một recipe cố định. Đóng góp dự kiến là vòng điều phối có ràng buộc vận hành, cùng bằng chứng về đánh đổi chất lượng–chi phí và độ tin cậy qua kiểm thử tích hợp trên môi trường thực nghiệm cùng kiến trúc production.

1 Bài toán, mục tiêu và phạm vi

1.1 Bài toán nghiên cứu

Data drift là thay đổi phân phối đầu vào $P(X)$ giữa reference và production. Cảnh báo drift không tự chứng minh mô hình suy giảm: thay đổi có thể xảy ra ở vùng mô hình vẫn dự đoán tốt. Ngược lại, sai số có thể tăng khi quan hệ $P(Y | X)$ thay đổi mà detector đầu vào không phát hiện. Do đó, giám sát drift và kiểm chứng chất lượng là hai nguồn bằng chứng bổ sung.

Ở thời điểm quyết định t , hệ thống quan sát batch production, prediction và xác suất của mô hình đang phục vụ; chỉ được đọc nhãn đã nhận hoặc được yêu cầu trong ngân sách. Cần lựa chọn KEEP, REQUEST_LABELS, HOLD hoặc RETRAIN. Khi không đủ bằng chứng, kết quả là chưa xác định; không gán mặc định mô hình khòe. Ước lượng không nhãn phụ thuộc giả định về shift và calibration, không bảo đảm nhận diện mọi suy giảm [?, ?].

1.2 Câu hỏi nghiên cứu và đóng góp dự kiến

Định vị MLOps: Đối tượng nghiên cứu là vòng điều phối từ giám sát đến cập nhật mô hình, không phải thuật toán ước lượng chất lượng mới. Một trigger đúng về thông kê vẫn có thể dẫn đến kết quả vận hành sai nếu dùng nhầm snapshot, chỉ nhãn hai lần khi retry hoặc áp dụng candidate cho champion đã thay đổi. Vì vậy, đề tài đánh giá cả hiệu quả quyết định và tính đúng đắn của quá trình thực thi.

1. **RQ1 – Hiệu quả vận hành:** Với cùng giới hạn ngân sách và recipe, kiểm chứng trước retraining đạt đánh đổi chất lượng, chi phí nhân, số lần huấn luyện và bỏ sót suy giảm như thế nào so với trigger theo drift, estimator hoặc audit theo lịch cố định?
2. **RQ2 – Độ tin cậy điều phối:** Vòng Continuous Training trên Argo Workflows/Kubeflow có bảo toàn ngân sách, tính nhất quán dữ liệu–phiên bản và điều kiện cập nhật khi có sự kiện lặp, lỗi worker/pod hoặc callback đến muộn không; cần bao lâu để phục hồi sau khi thành phần lỗi hoạt động trở lại?

Đóng góp gồm: (i) thiết kế và hiện thực vòng điều phối có kiểm chứng, trạng thái bền vững, quản lý ngân sách và truy vết; (ii) bộ đánh giá tái lập gồm replay chất lượng–chi phí và kiểm thử lỗi vận hành. Tích hợp hệ thống là đầu ra bắt buộc, không chỉ là minh họa cho notebook. Không đề xuất detector, estimator, thuật toán lấy mẫu hay lý thuyết hệ phân tán mới. Thành công được đánh giá bằng bằng chứng thực nghiệm và phân tích giới hạn, kể cả khi policy không vượt baseline.

1.3 Ranh giới thực hiện

- **Cốt lõi:** Hai dataset nhị phân tabular, một model family có xác suất lớp; CBPE chính, ATC đối chứng; mẫu nhãn ngẫu nhiên và một recipe full-retraining. Triển khai vòng tích hợp trên AWS/K3s với Argo Workflows, Kubeflow Training Operator và Prometheus/Grafana; Docker chỉ phục vụ kiểm thử local. Kiểm chứng retry, phục hồi và cập nhật phiên bản trên môi trường thực nghiệm cùng kiến trúc production.

- **Giả định nhân:** Dataset thực nghiệm có nhãn đầy đủ ở phía evaluator. Hệ thống chỉ nhận phần được tiết lộ theo ngân sách sau mỗi batch. Nhãn đúng, chỉ phí một đơn vị/mẫu; phần còn lại chưa được phép đọc. Yêu cầu nhãn được mô phỏng, không phải nghiên cứu hành vi người gán nhãn.
- **Ngoài phạm vi:** Taxonomy nguyên nhân, causal diagnosis, học policy bằng reinforcement learning, tối ưu sampler, incremental training, pseudo-labeling, nhãn trễ/sửa nhãn phức tạp và human study. Kế thừa Terraform/Ansible/GitOps, chỉ bổ sung cấu hình cần cho đề tài; không nghiên cứu high availability đa vùng, tối ưu autoscaling, distributed training hoặc canary production; không cam kết exactly-once toàn hệ thống.

Trường hợp không lấy được bất kỳ nhãn production nào chỉ dùng kiểm tra giới hạn giám sát. Không cam kết supervised retraining trên dữ liệu không nhãn. Tên dataset, cấu hình model, mức ngân sách và ngưỡng được chọn qua pilot ở tháng 1 rồi đóng băng trước test.

2 Cơ sở kế thừa và định vị đóng góp

Bảng 1: Các công trình trực tiếp phục vụ đề tài.

Công trình	Phần kế thừa	Giới hạn và cách sử dụng
ATC, ICLR 2022 [?]	Học ngưỡng confidence trên reference có nhãn để ước lượng accuracy trên target	Baseline nhẹ; hiệu quả phụ thuộc dạng shift, không chứng minh được suy giảm trong mọi môi trường
CBPE, Machine Learning 2026 [?]	Ước lượng confusion matrix và metric từ xác suất đã calibration	Estimator chính; calibration tốt trên reference không bảo đảm tốt trên production
MLDemon, AISTATS 2022 [?]	Kết hợp tín hiệu không nhãn và yêu cầu nhãn theo ngân sách để giám sát	Tiền lệ gần nhất cho policy; đối chiếu giả định và đánh giá thêm tác động lên retraining
Active Testing, ICML 2021 [?]	Phân biệt chọn mẫu đánh giá và chọn mẫu huấn luyện; xử lý thiên lệch lấy mẫu	Bản đầu dùng mẫu ngẫu nhiên để giới hạn độ phức tạp; không tuyên bố thuật toán active testing mới
Modyn, SIGMOD 2025 [?]	Tách trigger khỏi data-selection policy, đánh giá pipeline theo thời gian	Kế thừa cách tổ chức benchmark; giữ recipe cố định, không xây lại toàn bộ nền tảng

Các thành phần trên đã có trong nghiên cứu trước. Khóa luận kiểm chứng **giá trị của bước lấy thêm nhãn trong một vòng điều phối retraining**: mức giảm sai quyết định có bù được chi phí nhân và độ chậm phục hồi hay không, và vòng đó có thực thi đúng khi tác vụ lỗi hay không. Modyn là cơ sở đối chiếu về tổ chức pipeline; MLDemon là cơ sở về giám sát trong ngân sách. Các kỹ thuật snapshot, idempotency và lưu trạng thái bền vững là thực hành hệ thống được vận dụng, không được coi là phát minh mới. Đóng góp giới hạn ở thiết kế cụ thể và bằng chứng thực nghiệm; không khẳng định các paper trước chưa xử lý lỗi, hay đây là hệ thống đầu tiên kết hợp monitoring, nhãn và retraining.

3 Mức đáp ứng của hệ thống hiện tại

3.1 Cách đánh giá và phạm vi bằng chứng

Các năng lực ứng dụng E1–E8 được đối chiếu tại commit cde7dcb; phân hạ tầng/thực thi E9–E11 bổ sung tại 1022cff, ngày 07/09/2026. Đây là bằng chứng từ repo, không phải xác nhận trạng thái cluster. Đơn vị đánh giá là **năng lực cần cho đề tài**, không dùng thang trưởng thành của một khung kiến trúc MLOps.

- **Đầy đủ:** Có implementation đáp ứng năng lực được mô tả ở hàng tương ứng; có thể kế thừa sau kiểm tra tích hợp.
- **Một phần:** Có phần nền nhưng còn thiếu dữ liệu, contract hoặc xử lý cần cho thí nghiệm.
- **Thiếu:** Chưa tìm thấy implementation đáp ứng năng lực trong các luồng đã kiểm tra.

Mức “Đầy đủ” chỉ áp dụng cho yêu cầu hẹp trong bảng, không có nghĩa toàn bộ pipeline đã sẵn sàng hay được kiểm chứng production.

Bảng 2: Hiện trạng repo đối với đề tài đã chốt.

Năng lực cần có	Mức độ	Bảng chứng hiện tại và phần cần bổ sung
Lưu prediction và identity	Một phần	E1–E2 có prediction ID, thời gian, tenant/project/version và features. Cần kiểm soát tính đầy đủ của batch; producer gửi sự kiện thành công theo cơ chế best-effort.
Xác suất lớp cho CBPE/ATC	Một phần	E3 chỉ trả max confidence thang 0–100. Cần xác suất lớp dương 0–1, class mapping, decision threshold và calibration version.
So sánh phân phối, xuất báo cáo	Đầy đủ	E4 chạy Evidently với reference/production, sinh báo cáo và summary. Kế thừa detector, pin phiên bản và hiệu chỉnh ngưỡng trên development stream.
Kiểm tra dữ liệu đầu vào	Một phần	E4 kiểm tra thiếu cột, null cao; gộp cột thiếu vào drift tổng hợp. Cần tách lỗi contract khỏi statistical drift và chặn batch không hợp lệ.
Cửa sổ dữ liệu tái lập	Một phần	E4 lấy các bản ghi mới nhất theo version. Cần chốt khoảng thời gian, danh sách prediction ID và snapshot reference trước chạy.
Kích hoạt tác vụ giám sát	Một phần	E5 có khóa, watermark theo số lượng bản ghi và idempotency cho DriftRun. Chưa có audit nhân hay trigger retraining theo chất lượng.
Feedback và ngân sách nhân	Thiếu	E2/E5 chưa có nhận nhân gắn prediction và quản lý mục đích, quota, nhân đã tiết lộ. Cần LabelRequest, Feedback và budget ledger.
Ước lượng, kiểm chứng suy giảm	Thiếu	Chưa có CBPE/ATC, baseline chất lượng theo version, khoảng bất định và kiểm chứng trên mẫu nhân ngẫu nhiên.
Policy quyết định retraining	Thiếu	Chưa có vòng KEEP, REQUEST_LABELS, HOLD, RETRAIN với bằng chứng quyết định và giới hạn yêu cầu nhân.
Chạy job với snapshot đầu vào	Đầy đủ	E6 tạo TrainingJob, snapshot source/data và submit qua execution backend. Tái sử dụng để chạy recipe cố định.
Tạo training set từ production	Một phần	E6 nhận file training data nhưng mặc định lấy workspace asset mới nhất. Cần nối nhân và xuất snapshot từ pool production hợp lệ.
Đăng ký candidate sau build	Đầy đủ	E7 tạo ModelVersion và lưu artifact/metrics khi build thành công. Kế thừa lưu vết candidate.
Đánh giá candidate độc lập	Thiếu	Metrics được ingest chưa thay thế evaluator tin cậy. Cần holdout riêng, so sánh champion và quyết định pass/fail/hold.
Benchmark ngân sách nhân	Thiếu	Chưa có replay che nhân, policy đối chứng và thống kê chi phí/chất lượng của đề tài.
Phát tác vụ và lưu vết điều phối	Một phần	E5/E6 có khóa, trạng thái và dispatch sau commit; E8 có EventOutbox. Cần liên kết vào vòng maintenance, đối soát bước chưa hoàn tất và kiểm chứng khi worker lỗi.
Truy vết quyết định đến cập nhật	Một phần	E6/E7 có snapshot, job và version; thiếu liên kết evidence–policy–budget–gate–champion cho một quyết định end-to-end.
Tự động hóa triển khai AWS/K3s	Một phần	E9 có Terraform, Ansible và cây GitOps. Cần chốt cấu hình thực nghiệm, kiểm tra bootstrap và triển khai vòng CT mới; chưa xác nhận cluster từ mã nguồn.
Điều phối và huấn luyện production	Một phần	E10 chọn backend Argo, có WorkflowTemplate tạo PyTorchJob qua Training Operator. Cần nối policy, gate và đối soát retry vào vòng CT.
Giám sát vận hành vòng CT	Một phần	E11 có Prometheus/Grafana, ServiceMonitor và rule nền. Cần metrics, dashboard và cảnh báo cho ngân sách, tiến trình và lỗi maintenance.

3.2 Các điểm mã nguồn làm căn cứ

Các đường dẫn tính từ gốc repo, chỉ dẫn tới chức năng đã kiểm tra:

- **E1:** `services/model-server/src/index.py`, gửi event và endpoint predict.
- **E2:** `services/consumer/src/database.py`, production logs và automatic-drift outbox.
- **E3:** `services/machine-learning-serving/src/index.py`, xử lý predict_proba.
- **E4:** `services/evidently/src/main.py`, load_production_data và run_drift_analysis.
- **E5:** `services/control-plane/src/apps/drift/services/automatic.py`, tạo DriftRun.
- **E6:** `services/control-plane/src/apps/training/services/jobs.py`, tạo/submit job và snapshot.
- **E7:** `services/control-plane/src/apps/registry/services/versions.py`, đăng ký version.
- **E8:** `services/control-plane/src/apps/observability/services/outbox.py`, lưu sự kiện rồi yêu cầu phát sau commit; không tự chứng minh khả năng phục hồi của vòng maintenance mới.
- **E9:** `infra/main.tf`, `ansible/site.yml`, `ansible/roles/platform_core/tasks/main.yml`, `k8s/gitops/production/`; cập phát AWS, bootstrap K3s/Argo CD và bàn giao GitOps.
- **E10:** `k8s/apps/base/control-plane/configmap.yaml` chọn backend Argo; template training tại `k8s/argo/workflows/`; operator tại `k8s/addons/kubeflow-training/`.
- **E11:** Application monitoring tại `k8s/gitops/production/applications/addons/`; ServiceMonitor và PrometheusRule tại `k8s/platform/observability/`.

Nền tảng có các khối suy luận, drift report, training, lưu candidate và cấu hình triển khai production. Khối cần xây nằm ở **feedback, ngân sách và vòng điều phối có kiểm chứng**, tích hợp vào backend Argo/Kubeflow và monitoring hiện có. Kế thừa hạ tầng giúp giới hạn phạm vi, nhưng vẫn phải kiểm chứng triển khai AWS/K3s và luồng end-to-end; không suy ra bảo đảm phục hồi chỉ từ manifest, khóa hay outbox riêng lẻ.

4 Phương pháp đề xuất

4.1 Tổ chức dữ liệu và ngân sách nhân

Chia dữ liệu ban đầu thành các tập tách biệt: D_{train} để huấn luyện, D_{cal} để calibration/fit ATC và D_{ref} để lấy mốc drift, chất lượng. Mỗi version có estimator và mốc chất lượng riêng; sau khi thay champion phải fit lại estimator từ calibration data hợp lệ và ghi nhận mốc mới.

Batch production W_t được lưu prediction *trước* khi tiết lộ nhãn. Trước khi đọc nhãn, dùng seed để phân vai các mẫu thành pool kiểm chứng, huấn luyện và đánh giá candidate. Các pool là mẫu ngẫu nhiên của cùng batch và không trùng ID. Chỉ pool kiểm chứng được dùng kết luận chất lượng champion; pool huấn luyện dùng tạo dataset; pool đánh giá dành riêng cho candidate. Nhãn không được tái sử dụng sang pool khác trong thí nghiệm chính.

Đặt B là giới hạn số nhãn production được tiết lộ trên một episode. Ghi sổ ba khoản:

$$B_{\text{used}} = B_{\text{verify}} + B_{\text{train}} + B_{\text{gate}} \leq B.$$

Tất cả nhãn trả về, kể cả khi không retrain hoặc candidate bị loại, đều được tính; retry cùng ID không tính lại. Quota theo mục đích và lịch cấp nhãn huấn luyện được cố định trên development. Trong so sánh policy kiểm chứng, các phương pháp có cùng lịch và pool nhãn huấn luyện, tránh lợi ích chỉ do được cấp thêm training labels. Quota kiểm chứng gồm phần dành cho audit định kỳ và phần cho yêu cầu bổ sung; không tiêu hết phần audit vào các cảnh báo sớm.

4.2 Ước lượng và điều kiện xác nhận suy giảm

Evidently theo dõi X trên toàn batch hợp lệ. CBPE fit bằng reference/calibration có nhãn và ước lượng chất lượng từ xác suất lớp dương trên production [?]. ATC được fit độc lập trên cùng split làm baseline [?]. Cấu hình đầu tiên dùng **tỷ lệ lỗi** $e = 1 - \text{accuracy}$ làm metric quyết định chung vì cả hai estimator đều hỗ trợ; F1, recall và Brier là metric báo cáo bổ sung. Nếu lớp hiếm quá ít, không đưa kết luận riêng về recall của lớp đó.

Đặt e_{ref} là tỷ lệ lỗi champion trên D_{ref} , $\Delta_t = e_t - e_{\text{ref}}$ là mức suy giảm. Estimator cung cấp $\hat{\Delta}_t$ để gợi ý lấy nhãn, không phải ground truth. Với khoảng kiểm chứng $I_t = [L_t, U_t]$ từ mẫu nhãn thật:

- $L_t > \delta_{\text{harm}}$: xác nhận suy giảm vượt biên thực tiễn.
- $U_t \leq \delta_{\text{keep}}$: có bằng chứng giữ mô hình trong biên chấp nhận.
- Các trường hợp khác: chưa xác định; với $0 \leq \delta_{\text{keep}} \leq \delta_{\text{harm}}$.

Khoảng kiểm chứng phải tính bất định của cả mẫu production và mốc reference. Bản đầu cho phép tối đa hai lần đọc kết quả trong một cửa sổ, với cỡ mẫu dự kiến xác định trước. Lấy mẫu bằng tiền tố của một hoán vị ngẫu nhiên độc lập với nhãn; dùng cận cho sai số nhị phân và phân bổ mức lỗi thống kê giữa các lần đọc và mốc reference. Không dùng CI cố định rồi xem lặp vô hạn. Mức bảo đảm phát biểu theo cửa sổ; false-alarm toàn stream được đo riêng. Calibration và repeated testing là giới hạn cần báo cáo.

4.3 Policy kiểm chứng trong giới hạn ngân sách

Policy có tham số cố định: lịch audit k , cỡ mẫu ban đầu m_1 , cỡ mẫu bổ sung m_2 , ngưỡng drift, ngưỡng estimator, mức bất đồng và cooldown. Thực hiện theo thứ tự:

1. Kiểm tra contract, tính đầy đủ của batch và reference. Nếu lỗi, trả HOLD và ghi lý do; chưa dùng batch để retrain.
2. Tính drift và ước lượng chất lượng. Bắt đầu kiểm chứng khi đến kỳ audit, có drift hoặc estimator nghi suy giảm. Audit định kỳ vẫn chạy khi cả hai tín hiệu im lặng.
3. Nếu còn quota, yêu cầu mẫu ngẫu nhiên m_1 ở pool kiểm chứng. Khi I_t đã phân loại được, quyết định theo bảng ??.
4. Nếu chưa xác định hoặc estimator bất đồng rõ với nhãn thật, dùng tối đa một lượt bổ sung m_2 từ cùng hoán vị, rồi cập nhật khoảng kiểm chứng theo quy tắc hai lượt. Estimator không có quyền phủ quyết kết quả nhãn thật.
5. Khi hết quota, thiếu mẫu hoặc vẫn chưa xác định, trả HOLD, ghi hẹn audit tiếp theo. Không diễn giải HOLD thành mô hình khỏe.

Bảng 3: Quyết định sau kiểm chứng.

Hành động	Điều kiện và kết quả
KEEP	Khoảng kiểm chứng nằm trong biên chấp nhận; lưu bằng chứng, tiếp tục giám sát.
REQUEST_LABELS	Cần kiểm chứng và còn lượt/quota; chọn ID trước khi đọc nhãn, lưu chi phí.
HOLD	Chưa đủ bằng chứng, hết ngân sách, dữ liệu lỗi hoặc chưa đủ dữ liệu huấn luyện; có lý do và thời điểm xét lại.
RETRAIN	Nhãn thật xác nhận suy giảm, pool training đủ mẫu/lớp, còn quota đánh giá và không có job cùng champion đang chạy. Tạo candidate theo recipe cố định.

Phần cần kiểm chứng thực nghiệm là lợi ích của phân bổ audit theo bằng chứng so với lịch cố định. Không tự động học hay thay đổi tham số policy trên test. Không trộn F1 đo từ mẫu có nhãn với F1 ước lượng trên phần không nhãn bằng trung bình tùy ý.

4.4 Recipe retraining và đánh giá candidate

Recipe giữ nguyên model family, preprocessing, hyperparameters và phương thức huấn luyện từ đầu. Dataset gồm nhãn production thuộc pool training trong cửa sổ gần nhất, trộn tỷ lệ cố định từ D_{train} . Dùng lấy mẫu ngẫu nhiên/phân tầng với seed; cố định cửa sổ, tỷ lệ trộn và giới hạn số mẫu qua pilot. Dành calibration split không trùng training cho candidate. Khi thiếu dữ liệu/lớp, HOLD thay vì dùng nhãn giả.

Luồng thực thi là **TrainingJob** → **TrainingOutput** → **Build/Register candidate** → **EvaluationGate**. Gate dùng mẫu production riêng có nhãn được tính vào B_{gate} ; so sánh lỗi candidate và champion theo cặp trên cùng mẫu. Candidate chỉ pass khi lợi ích vượt biên đã định và không suy giảm quá biên trên historical holdout. Một gate dùng cỡ mẫu định trước; không mở rộng mẫu tùy kết quả trong MVP. Thiếu bằng chứng trả HOLD; fail không đổi champion. Mọi candidate đều được tính chi phí.

Trong replay, candidate pass được kích hoạt cho batch sau khi job/gate hoàn tất; champion cũ phục vụ trong thời gian chờ. Trên AWS/K3s, luồng Argo thực hiện cập nhật có kiểm soát và kiểm tra phiên bản phục vụ thực tế theo Mục ??.

Không đồng nhất đăng ký version với cải thiện chất lượng hay quyền tự động triển khai: chỉ cập nhật khi gate và các điều kiện vận hành đều đạt.

5 Thiết kế vòng điều phối Continuous Training

5.1 Phân công trách nhiệm

Consumer tiếp tục sở hữu ingestion và bảng prediction; Evidently giữ chức năng tạo drift report. Control Plane sở hữu trạng thái maintenance, yêu cầu nhân, ngân sách và quyết định; worker chỉ thực thi bước được giao. Phép tính CBPE/ATC, khoảng kiểm chứng và policy được tách thành thư viện thuần dùng chung trong replay và worker. PostgreSQL lưu trạng thái và quan hệ; object storage lưu snapshot/artifact; Redis chỉ dùng cho hàng đợi và log tạm. Thiết kế dưới đây là phần cần triển khai, không phải mô tả chức năng đã hoàn tất.

5.2 Môi trường triển khai và backend thực thi

Docker Compose chỉ dùng phát triển và kiểm thử local; không là backend triển khai cuối cùng hay bằng chứng thay thế cho Argo/Kubeflow. Môi trường đích là AWS/K3s, được dựng bằng Terraform và Ansible automation; cấu hình ứng dụng/operator được quản lý qua GitOps.

Bảng 4: Vai trò các thành phần trên môi trường production.

Thành phần	Trách nhiệm trong đề tài
Terraform / AWS	Cấp phát tài nguyên cloud và S3 từ cấu hình phiên bản hóa; ghi cấu hình tài nguyên và chi phí phục vụ thực nghiệm.
Ansible / K3s / Argo CD	Ansible chuẩn bị máy, bootstrap K3s và Argo CD; Argo CD đồng bộ add-on, ứng dụng và WorkflowTemplate từ Git. Không cài trùng operator bằng hai cơ chế.
Argo Events / Workflows	Nhận yêu cầu từ Control Plane, điều phối tác vụ drift, training, build, gate và cập nhật. Bổ sung bước gate, liên kết trạng thái và xử lý lỗi cho vòng CT mới.
Kubeflow Training Operator	Quản lý job huấn luyện do workflow tạo, theo dõi pod và trạng thái kết thúc. Kế thừa PyTorchJob CPU một replica của repo để chạy training-runner; không yêu cầu huấn luyện phân tán.
Prometheus / Grafana	Thu thập metrics hạ tầng và vòng CT, hiển thị dashboard, cảnh báo tác vụ kẹt hoặc lỗi; không thay thế policy hay sổ ngân sách trong DB.

Control Plane quyết định *có được thực thi hay không*; Argo quản lý tiến trình tác vụ, Training Operator quản lý tài nguyên huấn luyện. Lưu ảnh xạ MaintenanceRun–Workflow UID–TrainingJob–PyTorchJob UID để đối soát. Workflow hoàn tất chưa đồng nghĩa candidate đã qua gate hoặc đã phục vụ. Kế thừa namespace thực thi mlops-execution và namespace training user-jobs; không đổi mô hình sở hữu chỉ để phục vụ thí nghiệm.

5.3 Hợp đồng dữ liệu tối thiểu

Bảng 5: Các đối tượng tối thiểu cần bổ sung.

Đối tượng	Thông tin và ràng buộc chính
PredictionRecord mở rộng	Prediction ID, tenant/project/version, event time, features, predicted class, xác suất lớp dương, class mapping và calibration version. Giữ tương thích trường confidence cũ.
LabelRequest / Feedback	ID mẫu, mục đích verify/train/gate, seed, thời điểm yêu cầu/nhận nhân, trạng thái và chi phí. Chỉ nối feedback khi identity khớp.
BudgetLedger	Quota và khoản đã giữ/chốt theo episode, mục đích và request; retry không chi hai lần, không vượt ngân sách khi có request đồng thời.
EvidenceWindow	Danh sách ID và reference snapshot, champion, kết quả drift/estimator/audit, số nhân và thời điểm chốt. Nhân chưa được cấp không xuất hiện trong view của policy.
MaintenanceDecision	Action, reason, phiên bản policy, evidence snapshot, các chi phí liên quan và TrainingJob nếu có; lưu cả KEEP/HOLD.
MaintenanceRun	Gắn decision với các bước job/build/gate/update; lưu trạng thái, attempt, thời hạn chờ, lỗi và khóa chống xử lý lặp. Tách trạng thái thực thi khỏi action của policy.

Đối tượng	Thông tin và ràng buộc chính
DatasetSnapshot / EvaluationGate	Manifest train/calibration/gate, recipe và hash dữ liệu; kết quả so sánh theo cặp, trạng thái pass/fail/hold và candidate version.

5.4 Vòng đời và điều kiện chuyển trạng thái

Luồng tích hợp gồm hai chặng: thu thập prediction, chốt evidence, nhận feedback và lưu decision; sau đó chạy training, build/register, gate và cập nhật có kiểm soát. Action KEEP/HOLD/RETRAIN không thay thế trạng thái của job. Mỗi lượt lưu champion kỳ vọng tại thời điểm chốt bằng chứng và quản lý các bước như Bảng ??.

Bảng 6: Trạng thái điều phối đề xuất và điều kiện tiến bước.

Trạng thái	Điều kiện và kết quả
Chờ nhân / ra quyết định	Chốt evidence và giữ quota trước yêu cầu nhân; lưu KEEP/HOLD hoặc cho phép tạo lượt RETRAIN.
Chờ chạy / đang thực thi	Có snapshot hợp lệ, quota đã giữ và khóa lượt đang hoạt động; theo dõi riêng training và build. Thành công chỉ tạo candidate.
Chờ đánh giá	Artifact sẵn sàng, gate có đủ nhân độc lập; pass chuyển sang chờ cập nhật, fail/hold giữ champion.
Chờ cập nhật / đã áp dụng	Gate pass và champion hiện tại vẫn khớp champion kỳ vọng; chỉ ghi đã áp dụng sau khi runtime K3s sẵn sàng và kiểm tra inference đúng phiên bản.
Lỗi / hết hạn / bị thay thế	Ghi bước lỗi, attempt và lý do; callback cũ không phục hồi trạng thái kết thúc. Bằng chứng cũ không được dùng thay champion mới.

MVP chỉ cho phép một lượt cập nhật đang hoạt động trên mỗi project; các cửa sổ giám sát vẫn được ghi nhận nhưng không tạo lượt huấn luyện cạnh tranh. Khi champion đổi trong lúc candidate đang chạy, lượt cũ được đánh dấu bị thay thế; muốn đánh giá lại phải tạo quyết định mới. Thiếu nhân hoặc hết ngân sách là HOLD có lý do, khác với lỗi thực thi cần retry. Người vận hành có thể xem xét HOLD/lỗi; thao tác thủ công vẫn phải ghi vết và không bỏ qua gate.

5.5 Bất biến và phục hồi khi tác vụ lỗi

- Ngân sách nhất quán:** Khóa ledger và giữ quota trong transaction; tổng đã chi và đang giữ không vượt trần. Chốt chi phí một lần khi nhân được tiết lộ. Chỉ hoàn quota khi xác nhận yêu cầu chưa tiết lộ nhân; timeout chưa rõ kết quả phải đối soát, không hoàn ngay.
- Xử lý lặp có cùng kết quả:** Dùng khóa theo decision/bước và ràng buộc duy nhất trong DB; retry vận chuyển không tạo thêm logical job, khoản chi hay cập nhật. Mỗi attempt lưu riêng; chỉ retry huấn luyện sau khi xác nhận attempt trước đã dừng. Khi chưa rõ runtime còn chạy hay không, giữ trạng thái chờ đối soát.
- Không mất bước sau commit:** Ghi trạng thái và sự kiện cần phát trong cùng transaction, tái sử dụng mẫu outbox. Worker đối soát định kỳ các bước chưa hoàn tất với DB và runtime; không chỉ dựa vào thông báo hàng đợi hoặc callback. Retry có giới hạn và thời hạn; quá giới hạn chuyển lỗi có thể kiểm tra.
- Không cập nhật sai phiên bản:** Gate gắn với candidate, champion và snapshot cụ thể. Kiểm tra lại champion kỳ vọng khi áp dụng; callback trùng/đảo thứ tự/đến muộn không được vượt bước. Nếu khởi động candidate thất bại, giữ champion cũ và không ghi nhận cập nhật thành công.

Phân định retry theo tầng: Argo xử lý bước tác vụ theo cấu hình giới hạn, Training Operator báo trạng thái job/pod; Control Plane đối soát và quyết định tạo attempt mới. Không để cả ba tầng tự tạo lại cùng một lượt huấn luyện. Khi mất callback, truy vấn Workflow/PyTorchJob theo UID đã lưu trước khi phát lại; trạng thái không rõ không được coi là thất bại đã kết thúc. Cấu hình timeout và retry được phiên bản hóa cùng WorkflowTemplate. Mục tiêu là hiệu ứng nghiệp vụ không lặp trên các tình huống đã kiểm thử, không phải bảo đảm exactly-once toàn hệ thống. Lỗi hệ thống lưu trữ mất dữ liệu vĩnh viễn và sự cố nhiều vùng nằm ngoài phạm vi. Feedback và evidence giữ giới hạn tenant/project/version theo contract hiện có; không mở rộng cơ chế phân quyền.

5.6 Truy vết và quan sát vận hành

Mỗi decision phải truy ngược được prediction ID, nhãn đã dùng, reference và manifest các pool; truy xuôi được job, build, gate và lần cập nhật. Manifest lưu hash dữ liệu, snapshot mã nguồn, cấu hình recipe/policy, seed, phiên bản estimator và môi trường thực thi. Chốt manifest trước dispatch; workspace thay đổi không được làm đổi dữ liệu của lượt đang chạy. Khi artifact đã bị xóa, đánh dấu không còn khả năng tái lập thay vì coi hash là đủ để chạy lại.

Log có cấu trúc dùng cùng mã maintenance để nối các bước; trạng thái và lý do quyết định lưu trong DB. Bổ sung metrics qua ServiceMonitor vào Prometheus và một dashboard Grafana gọn: tuổi tác vụ chờ, thời lượng từng bước, số retry/lỗi, nhãn đã giữ/chi và gate pass/fail/hold; đối chiếu CPU/RAM và pod restart. UUID và prediction ID nằm trong DB/log, không đưa thành metric label gây cardinality cao. Truy vết chi tiết và lịch sử champion được tra cứu từ Control Plane.

Cảnh báo gồm workflow không tiến triển quá deadline, job thất bại và quota sắp cạn; cảnh báo vận hành không trực tiếp kích hoạt retraining. Bằng chứng drift/chất lượng vẫn phải qua policy. Demo trên AWS/K3s phải thể hiện KEEP, lấy thêm nhãn, một lượt cập nhật thành công và một lượt lỗi được phục hồi hoặc kết thúc có lý do, đồng thời tra được diễn biến trên Grafana và nhật ký liên kết.

6 Thiết kế thực nghiệm và tiêu chí đánh giá

Thiết kế gồm hai phần: **RQ1** dùng replay có đối chứng để đo chất lượng–chi phí; **RQ2** chạy luồng tích hợp Argo/Kubeflow trên AWS/K3s và chủ động gây lỗi để kiểm tra bất biến, phục hồi và truy vết. Docker dùng kiểm thử local trước triển khai. Notebook/replay không thay thế bằng chứng thực thi trên backend production.

6.1 Dữ liệu, kịch bản và chống rò rỉ

Chọn hai dataset tabular nhị phân có nguồn/giấy phép rõ ràng, đủ số mẫu và cả hai lớp ở các cửa sổ; ưu tiên ít nhất một dataset có thời gian hoặc nhóm miền tự nhiên. Bộ sinh dữ liệu có nhãn phục vụ kiểm soát cơ chế drift. Dùng một model family có xác suất lớp; đề xuất bắt đầu từ logistic regression. Một model phi tuyến chỉ là kiểm tra độ nhạy nếu còn thời gian.

Chia riêng initial training/calibration/reference, development episodes để chọn cấu hình và test episodes để báo cáo. Nếu có thời gian/entity, chia theo chúng trước khi tạo batch; không đưa cùng thực thể vào hai phía trái với mục tiêu đánh giá. Giữ bộ seed test riêng. Future labels và cơ chế tạo drift chỉ thuộc evaluator, không thuộc estimator/policy.

Bảng 7: Kịch bản và vai trò trong thực nghiệm.

Kịch bản	Mục đích
Ổn định	Đo false-alarm, chi phí giám sát và việc tiêu ngân sách khi không có suy giảm.
Covariate shift với nhiều mức tác động	Đo drift có/không đi kèm suy giảm và khả năng phục hồi. Tạo shift bằng lấy mẫu theo X trong khi giữ cơ chế sinh nhãn; không tùy ý sửa X rồi giữ nhãn cũ.
Phân phối quay lại	Đo tác động giữ dữ liệu lịch sử trong recipe cố định và chất lượng xuyên nhiều đợt cập nhật.
Concept drift, calibration không còn phù hợp	Kiểm tra giới hạn estimator; audit định kỳ có phát hiện suy giảm bị tín hiệu không nhãn bỏ sót không.
Nhãn quan sát có thiên lệch	Stress test khi mẫu nhãn không còn đại diện; báo cáo sai số và không suy rộng bảo đảm của mẫu ngẫu nhiên.
Không lấy thêm được nhãn	Đo giới hạn ước lượng/HOLD; không xem là bài toán supervised retraining đã được giải quyết.

Ba hàng đầu là kịch bản chính. Trong ba hàng sau, ưu tiên một kịch bản estimator mất tin cậy và một kịch bản không có nhãn; nhãn thiên lệch là mở rộng. Không chạy tích Descartes của mọi drift, lỗi hệ thống và ngân sách. Độ có hại được đo trên champion của từng policy, không gán cứng từ tên drift. Trên dữ liệu thực, không tuyên bố suy giảm có nguyên nhân nhân quả đã biết. Không lọc test episodes theo việc phương pháp đề xuất thắng.

6.2 Các phương pháp đối chứng

Bảng 8: Baseline dùng cùng recipe và gate, trừ đối chứng không cập nhật.

Mã	Phương pháp	Vai trò
B0	Giữ nguyên mô hình	Đường chất lượng và chi phí khi không retrain.
B1	Drift-only trigger	Cảnh báo Evidently đủ để thử retrain, không audit trước trigger; gate vẫn dùng nhân riêng.
B2	Estimator-only trigger	CBPE hoặc ATC nghi suy giảm thì thử retrain; đo hậu quả của không kiểm chứng trước trigger.
B3	Audit theo lịch cố định	Lấy nhãn ngẫu nhiên theo lịch, quyết định từ chất lượng đo được; không dùng drift/estimator để phân bổ nhãn.
B4	Policy đề xuất	Audit định kỳ kết hợp kiểm chứng theo tín hiệu và một lượt bổ sung; quyết định theo Mục 4.
B5	Đối chứng đầy đủ nhãn	Có mọi nhãn ở pool kiểm chứng; giữ cùng training pool và recipe để đo khoảng cách thông tin. Báo cáo chi phí nhãn riêng, không coi là baseline cùng ngân sách.

B1–B4 cùng giới hạn ngân sách tổng, quota theo mục đích, giới hạn compute, lịch nhãn training và điều kiện gate. Ngân sách bằng nhau không có nghĩa số nhãn thực dùng bằng nhau: báo cáo cả quota và chi tiêu, kèm đường chất lượng theo chi phí thực. Không cấp nhãn gate miễn phí cho B1/B2. B3 là đối chứng performance-only quan trọng vì nhãn có thể đủ hữu ích ngay cả khi không dùng drift.

Thực nghiệm estimator CBPE/ATC được đánh giá gọn trên cùng prediction stream để giải thích sai quyết định. Với B3/B4, trước hết dùng cùng champion cố định; sau đó chạy replay có retraining để đo hiệu quả toàn vòng. Ablation chính của B4 lần lượt bỏ tín hiệu drift, bỏ audit định kỳ và bỏ lượt nhãn bổ sung; giữ các yếu tố khác tương đương. Tái hiện thêm MLDemon hoặc PAPE chỉ làm sau các kiểm thử vận hành bắt buộc; không gọi bản đơn giản hóa là thuật toán gốc và không tuyên bố vượt state of the art nếu chưa so sánh.

6.3 Chỉ số và cách đo đánh đổi

Bảng 9: Định nghĩa chỉ số dùng trong báo cáo.

Nhóm	Cách đo
Ước lượng chất lượng	MAE giữa tỷ lệ lỗi ước lượng và tỷ lệ lỗi đầy đủ của batch; độ phủ/độ rộng khoảng nếu phương pháp cung cấp khoảng hợp lệ.
Sai quyết định	Tỷ lệ trigger khi Δ_t đầy đủ không vượt biên có hại; tỷ lệ episode suy giảm không được kích hoạt trong deadline đã định. Tách HOLD, thiếu training data và hết ngân sách khỏi KEEP có bằng chứng.
Chất lượng hệ thống	Tỷ lệ lỗi theo batch và tổng lỗi theo số prediction; F1/recall bổ sung; thời gian đến lúc chất lượng trở lại biên chấp nhận. Episode chưa phục hồi khi hết test được ghi chưa phục hồi.
Chi phí nhãn	Số ID được tiết lộ theo verify/train/gate, tỷ lệ trên production và số request. Nhãn ẩn dùng riêng chấm benchmark không thuộc thông tin online.
Chi phí tính toán	Thời gian/CPU-hours cho estimator, chọn mẫu, training, build và gate; số mẫu học, số job và số candidate bị loại. Tách phép đo thật khỏi độ trễ mô phỏng.
Lợi ích retraining	Chênh lệch chất lượng candidate–champion trên cùng gate và trên batch tương lai; chất lượng historical holdout. Phân biệt trigger hợp lý với candidate thực sự cải thiện.

Đề đo đánh đổi KEEP/RETRAIN, trên một tập thời điểm được chọn trước khi xem nhãn test, clone cùng checkpoint thành hai nhánh: giữ mô hình và chạy recipe với cùng nhãn training được phép. So sánh trên cùng các batch tương lai; thông tin này chỉ để phân tích hồi cứu, không quay lại policy. Chi phí các nhánh phân tích được báo cáo riêng, không trộn với chi phí vận hành của phương pháp.

6.4 Protocol thống kê và điều kiện kết luận

Thử một số mức ngân sách, chẳng hạn 5%, 10%, 20% số mẫu production và đối chứng đầy đủ nhân; đây là lưới pilot, chỉ giữ cấu hình đủ mẫu kiểm chứng/huấn luyện/gate. Cố định ngưỡng, phân bổ quota, batch size, số lượt audit, recipe, seed và thời hạn phục hồi trước test. Ưu tiên 10 seed độc lập nếu pilot xác nhận đủ tài nguyên; chốt số lần lặp trước chạy chính thức.

So sánh theo cặp trên cùng dataset/episode/seed, bootstrap ở cấp episode hoặc block thời gian, không coi các batch phụ thuộc là mẫu độc lập. Báo cáo khoảng tin cậy và mức khác biệt thực tiễn; hiệu chỉnh khi kiểm định nhiều so sánh chính. Kết luận tiết kiệm phải đi kèm chất lượng nằm trong biên non-inferiority đã chốt; $p > 0.05$ không chứng minh tương đương. Trình bày riêng giới hạn do lớp hiếm, selection bias, calibration sai và repeated testing. Không hứa tỷ lệ tiết kiệm hoặc mức accuracy cải thiện trước khi thực nghiệm.

6.5 Kiểm thử độ tin cậy của vòng điều phối

Kiểm thử hai tầng, dùng một dataset nhỏ cố định cho RQ2. **Local:** unit tests và integration tests trên Docker kiểm tra policy, khóa, persistence, ngân sách và callback bằng dependency mô phỏng hoặc dịch vụ local. **AWS/K3s:** chạy end-to-end thật qua Argo Events/Workflows, Kubeflow Training Operator, S3 và monitoring. Dùng môi trường thực nghiệm riêng cùng kiến trúc production, không gây lỗi trên dịch vụ đang phục vụ người dùng.

Chèn lỗi tại các mốc trước/sau commit, dispatch và ghi kết quả; lặp lại với cùng dữ liệu và so sánh lượt không lỗi. Toàn bộ nhóm tình huống ở Bảng ?? cần có kiểm thử tích hợp trên AWS/K3s: restart pod worker theo điểm kiểm soát, tri hoãn/lập callback bằng test harness và dùng job lỗi chủ đích. Không cần dùng node, phá storage hoặc gián đoạn controller dùng chung. Khóa cấu hình CPU/RAM, số replica, image, WorkflowTemplate và phiên bản operator; lưu commit Terraform/Ansible/GitOps để tái lập.

Bảng 10: Tình huống lỗi và kết quả vận hành cần kiểm chứng.

Tình huống	Kết quả kỳ vọng
Nhân gửi trùng; yêu cầu đồng thời gần hết quota	Mỗi ID được ghi chỉ một lần; tổng đã ghi và đang giữ không vượt trần; yêu cầu thiếu quota có lý do từ chối/HOLD.
Worker/pod dừng sau commit hoặc sau dispatch	Đối soát DB với Workflow/PyTorchJob; không tạo logical job trùng; attempt chưa rõ kết quả không được chạy lại mù quáng.
Callback trùng, đảo thứ tự hoặc quá hạn	Không vượt bước, không ghi thêm candidate/cập nhật cho cùng kết quả và không khôi phục trạng thái đã kết thúc.
Training, build hoặc gate lỗi/timeout	Retry có giới hạn; trường hợp không phục hồi kết thúc với lý do; champion cũ vẫn phục vụ. Gate fail/hold không được tự chuyển thành pass.
Workspace hoặc champion đổi giữa các bước	Job vẫn dùng snapshot đã chốt; candidate dựa trên champion cũ bị chặn cập nhật và được ghi là bị thay thế.
Candidate không đạt gate hoặc không khởi động được	Không ghi cập nhật thành công; truy vết được nguyên nhân và phiên bản thực tế vẫn là champion cũ.

Chỉ số RQ2: Số vi phạm ngân sách; số logical job/cập nhật trùng; số chuyển trạng thái sai hoặc cập nhật khi gate chưa pass; tỷ lệ decision có đủ chuỗi truy vết; số lượt phục hồi và số lượt kết thúc lỗi có lý do. Đo thời gian từ khi dependency hoạt động trở lại đến khi lượt tiếp tục hoặc kết thúc có kiểm soát, báo cáo median, p95 và số mẫu. Nếu mất sự kiện không phát hiện được, phải ghi là lỗi thay vì loại khỏi mẫu.

Chi phí điều phối: Tách thời gian tính estimator/huấn luyện khỏi thời gian tạo cửa sổ, ghi ledger, chờ dispatch/scheduling, gate và áp dụng phiên bản. So sánh cùng workflow có/không chèn lỗi để đo chi phí retry, độ trễ tăng thêm và khả năng tiếp tục phục vụ của champion. Lấy timestamp từ DB/Workflow và metrics Prometheus; Grafana dùng quan sát, không là nguồn duy nhất của kết quả. Báo cáo tài nguyên pod, cấu hình

node, thời gian sử dụng và chi phí AWS; tách chi phí nền cluster/storage khỏi chi phí job, đánh dấu khoản ước tính. Không suy rộng cấu hình thực nghiệm nhỏ thành SLA production hay benchmark thông lượng lớn.

Nghiệm thu: Có bằng chứng triển khai Terraform/Ansible/GitOps và luồng training–gate–cập nhật chạy trên AWS/K3s; dashboard/cảnh báo nhận được metrics thực. Các trường hợp bắt buộc phải không có vi phạm bất biến trong số lượt đã chạy; mọi decision có lineage đầy đủ hoặc được ghi rõ artifact thiếu, và mọi lượt lỗi có trạng thái/lý do tra cứu được. Cố định số lần lặp, timeout và retry cap qua pilot; báo cáo toàn bộ lỗi phát hiện, kể cả đã sửa. Local pass không thay thế nghiệm thu production backend. Đây là bằng chứng trên tập kiểm thử hữu hạn, không phải chứng minh độ tin cậy tuyệt đối.

7 Kế hoạch 5 tháng, đầu ra và giới hạn

Bảng 11: Các mốc triển khai và tiêu chí nghiệm thu.

Tháng	Công việc trọng tâm	Đầu ra kiểm chứng được
1	Chọn dataset/model, pilot ngân sách; thiết kế trạng thái; rà soát Terraform/Ansible và chi phí AWS	Protocol version 1; cấu hình AWS/K3s, quota cloud; smoke test Argo/Kubeflow và monitoring
2	Replay che nhãn, B0–B4; feedback/window, ledger và manifest; kiểm thử Docker local	Unit/integration tests policy, trạng thái và quota; replay tái lập, contract tương thích backend Argo
3	Nối training, build/gate, cập nhật và đối soát trên AWS/K3s; metrics Prometheus/Grafana	Luồng end-to-end thật, trace DB–Workflow–job; dashboard và cảnh báo tối thiểu
4	Benchmark RQ1, ablation gọn; ma trận lỗi RQ2 trên môi trường AWS/K3s thực nghiệm	Bảng chất lượng–chi phí, bất biến, phục hồi và tài nguyên/chi phí AWS; phân tích thất bại
5	Hoàn thiện phân tích, báo cáo, demo AWS/K3s; dự phòng sửa lỗi	Mã nguồn, cấu hình triển khai, manifest, script chạy lại; kết quả và giới hạn có thể kiểm tra

Đầu ra bắt buộc: Vòng Continuous Training trên AWS/K3s qua Argo Workflows/Kubeflow, có kiểm chứng và phục hồi; cấu hình Terraform/Ansible/GitOps tái lập; metrics/dashboard Prometheus/Grafana; đặc tả trạng thái, manifest và log truy vết; benchmark hai dataset và kiểm thử lỗi theo Bảng ??; báo cáo trả lời hai RQ. PAPE, model family thứ hai, nhãn thiên lệch và giao diện nâng cao chỉ làm sau các đầu ra này. Không thay phần tích hợp AWS/K3s bằng notebook hoặc Docker khi thiếu thời gian.

Phân công dự kiến: Hoàng phụ trách policy, estimator, replay và phân tích chất lượng–chi phí; Thái phụ trách trạng thái/ledger, tích hợp Argo/Kubeflow, triển khai AWS/K3s và monitoring. Hai thành viên cùng chốt contract, tích hợp gate/cập nhật và kiểm thử lỗi; không tách thành một phần AI và một phần hạ tầng không liên kết.

Rủi ro trọng yếu và xử lý:

- **Nhân quá ít:** Pilot cỡ mẫu/độ rộng khoảng; chấp nhận HOLD, không bỏ bất định để có kết luận.
- **Estimator mất calibration:** Audit định kỳ độc lập cảnh báo, fit theo version; kiểm tra concept drift như trường hợp giới hạn.
- **Chi phí thiếu hoặc leakage:** Tính đủ ba pool nhãn và candidate bị loại; tách train/calibration/gate, khóa nhãn ẩn.
- **Vượt phạm vi:** Một recipe/model family, nhãn mô phỏng, một cấu hình AWS/K3s CPU; kế thừa IaC/operator. Cắt mở rộng trước kiểm thử bắt buộc.
- **Chi phí/quota AWS:** Smoke test sớm, chốt trần chi phí và gom lịch chạy; giảm ma trận phụ khi thiếu tài nguyên. Giữ artifact tái lập; không thay bằng chứng cloud bằng Docker.
- **Lỗi bất đồng bộ:** Cố định điểm chèn lỗi, lưu attempt/correlation ID; đối soát runtime trước retry, không coi timeout là đã dừng.
- **Policy không hơn baseline:** Báo cáo thất bại và đánh đổi; không sửa ngưỡng theo final test hay suy rộng từ hai dataset.

Nghiệm thu xác nhận phạm vi đã kiểm thử, không đồng nghĩa chẩn đoán mọi nguyên nhân suy giảm hay bảo đảm vận hành tự trị trong production.

Tài liệu tham khảo

- [1] S. Garg, S. Balakrishnan, Z. C. Lipton, B. Neyshabur, and H. Sedghi, “Leveraging unlabeled data to predict out-of-distribution performance,” *ICLR*, 2022. <https://arxiv.org/abs/2201.04234>.
- [2] J. Kivimäki, J. Białek, W. Kuberski, and J. K. Nurminen, “Performance estimation in binary classification using calibrated confidence,” *Machine Learning*, vol. 115, Art. 67, 2026. <https://doi.org/10.1007/s10994-025-06970-3>.
- [3] T. Ginart, M. J. Zhang, and J. Zou, “MLDemon: Deployment monitoring for machine learning systems,” *AISTATS*, PMLR, vol. 151, pp. 3962–3997, 2022. <https://proceedings.mlr.press/v151/ginart22a.html>.
- [4] J. Kossen, S. Farquhar, Y. Gal, and T. Rainforth, “Active testing: Sample-efficient model evaluation,” *ICML*, PMLR, vol. 139, pp. 5753–5763, 2021. <https://proceedings.mlr.press/v139/kossen21a.html>.
- [5] M. Böther, T. Robroek, V. Gsteiger, R. Holzinger, X. Ma, P. Tözün, and A. Klimovic, “Modyn: Data-centric machine learning pipeline orchestration,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, Art. 55, 2025. <https://doi.org/10.1145/3709705>.
- [6] J. Białek, J. Kivimäki, W. Kuberski, and N. Perrakis, “Estimating model performance under covariate shift without labels,” *NeurIPS*, vol. 38, 2025. <https://arxiv.org/abs/2401.08348>.